Let's search for documents where the movie title contains a word starting with *god*.

Listing 9.15 Wildcard search in action

```
GET movies/_search
{
  "query": {
    "wildcard": {
      "title": {
        "value": "god*"
      }
    }
  }
}
```

The wildcard query type

Searches the value field with a wildcard operator

Three movies (*The Godfather*, *The Godfather II*, and *City of God*) should be returned for this `wildcard` query. Of course, any movie (*Godzilla*, *God's Waiting List*, etc.) can also be fetched because we expect all titles that have the prefix *god* (those movies are not in our index, but if they were, the query would return them, too).

Wildcard queries on text fields

The query in listing 9.15 is run on the `title` field, which is a `text` data type. Because term-level queries are not analyzed, we use lowercase "god". Also, the `title` field was indexed with the `standard` analyzer, which, by default, lowercased the word.

If you want to use a `keyword` field instead of a `text` field, change `title` to `title.original` (the `title.original` field is defined as `keyword` in our `movies` schema) and run the query with the value "The God*". But if you omit "The" from "The God*" and run the query, you won't get any results. Because `title.original` is a `keyword` typed field, the value is persisted during indexing without any text analysis.

We can tweak our queries by placing wildcards anywhere in a word. For example, the query "g*d" fetches two movies from our stash: *The Good, the Bad, and the Ugly* and *City of God*. If we want to find the match of a given query criterion in a return document, we can use highlighting (discussed in chapter 8). The following listing shows this approach.

Listing 9.16 Searching with a wildcard operator in a word

```
GET movies/_search
{
  "_source": false,
  "query": {
    "wildcard": {
      "title": {
        "value": "g*d"
      }
    }
  }
```

Wildcard operator between the letters of a word